Composition API也叫组合式API，是Vue3.x的新特性。

Composition API也叫组合式API，是Vue3.x的新特性。

通过创建 Vue 组件，我们可以将接口的可重复部分及其功能提取到可重用的代码段中。仅此一项就可以使我们的应用程序在可维护性和灵活性方面走得更远。然而，我们的经验已经证明，光靠这一点可能是不够的，尤其是当你的应用程序变得非常大的时候——想想几百个组件。在处理如此大的应用程序时，共享和重用代码变得尤为重要。

composition-API：

[setup,ref,reactive,toRefs,computed,watch,watchEffect,"生命周期"Hook]

setup

setup() 函数是 vue3 中，专门为组件提供的新属性。它为我们使用 vue3 的 Composition API 新特性提供了统一的入口。

data、methods、computed、watch、watchEffect都是写在setup中

```
<template>
<!-- 在vue3.0 中的template模板 可以有多个根节点 -->
</template>
setup(props,context){
   //props : 接收组件props属性
   //context : 上下文对象，上下文对象包含一些属性，attrs、slots、emit(常用的三个)，
   //这些属性在vue2.x中需要通过this才能访问
   //在vue3.0中 setup中 不能访问vue2中的this对象，只能在setup中通过context访问
this中最常用的几个属性
}

//子组件Home
<template>
   <!-- 在vue3.0 中的template模板 可以有多个根节点 -->
</template>
      props:{
         title:{
            type:String,
            default:"这是一个title"
         }
      }
//父组件App
<template>
   <!-- 在 3.x 中，组件现在可以有多个根节点！ -->
   <!-- <HelloWorld></HelloWorld> -->
   <Home title="这是app组件传递过来的值"></Home>
</template>
```

ref

ref 接受一个内部值并返回一个响应式且可变的 ref 对象。ref 对象具有指向内部值的单个 property .value。

```
import {ref} from "vue"
const count = ref(0)
//如此以来在页面上定义一个按钮 count++ 就是响应式了
```

如果将对象分配为 ref 值，则可以通过 reactive 方法使该对象具有高度的响应式。

reactive

reactive() 函数接收一个普通对象，返回一个响应式的数据对象。响应式转换是“深层”的——它影响所有嵌套 property。在基于 ES2015 Proxy 的实现中，返回的代理是不等于原始对象。建议只使用响应式代理，避免依赖原始对象。

```
//定义对象、数组都可以
const data = reactive({
    test:"测试",
    user:{
        name:"木道人",
        sex:"男"
    },
    list:["特伦斯","追梦","请叫我小平头"]
})
```

ref ： 比较推荐定义基本数据类型

reactive ： 推荐定义复杂类型的数据时使用

toRefs

toRefs 将响应式对象转换为普通对象，其中结果对象的每个 property 都是指向原始对象相应property 的ref。普通对象的属性都一个ref，和响应式的属性 一 一对应。

当从合成函数返回响应式对象时，toRefs 非常有用，这样消费组件就可以在不丢失响应性的情况下对返回的对象进行分解/扩散：

```
//下面使用解构响应式对象数据，所以可以直接使用属性名
姓：<input type="text" v-model="firstName">
名：<input type="text" v-model="lastName">

 const user = reactive({
    firstName:"",
    lastName:""
})

const fullName =computed(()=>{
    return user.firstName + " " + user.lastName
})
const getFullName= ()=> {
        console.log(fullName.value)
 }
// 因为模板中要使用这些变量和方法，都需要调用，所以需要return出去
return{fullName,getFullName,
        ...toRefs(user)}
```